

GEMINI

Helix OTA — GEMINI.md

INHERITED FROM constitution/GEMINI.md

All rules in `constitution/GEMINI.md` and the `constitution/Constitution.md` it references apply unconditionally to this project. Project-specific rules below extend them — they do NOT weaken or override any universal clause.

When this file disagrees with the constitution submodule, the constitution wins.

Project overview

Helix OTA is an enterprise-grade over-the-air update system: a custom Go control plane (Gin modular monolith) driving native Android A/B updates (`update_engine` + AVB/dm-verity + auto-rollback) for RK3588 / Orange Pi 5 Max targets, with a roadmap to Linux, Windows, and other operating systems. Reusable building bricks live in `submodules/` (six `ota-*` modules) and the `dev/runtime` infrastructure in `containers/`.

Project-specific MANDATORY constraints

Build / packaging

- Server: Go module `github.com/HelixDevelopment/helix_ota/server` (`server/`). `go build ./...`, `gofmt`, `go vet` MUST be clean.
- Submodule Go bricks build + test from their own module roots under `submodules/`.
- Android bricks (`ota-update-engine-bridge`, `ota-android-agent`) build with AGP 8.5.2 + plain `kotlin.android` on Gradle 9.5 (Kotlin MPP does NOT build on Gradle 9.5 — see CONTINUATION handoff).
- Documentation source of truth is Markdown + Mermaid; PDF/HTML/DOCX and `draw.io`/SVG/UML/PNG are generated artifacts (`scripts/export_docs.sh`).

Test / verification

- Server: `cd server && go test ./...` (httptest integration suite).
- Inheritance gate (this wiring): `bash tests/inheritance_gate.sh`.
- Full inheritance + paired-mutation proof: `bash tests/test_constitution_inheritance.sh`.

Deployment / flash / publish

- Multi-upstream: the parent and every owned submodule push to all four upstreams (GitHub primary + GitLab + GitFlic + GitVerse). The parent's origin remote fans out pushes to all four.

Feature video recording + Status doc mandates (§11.4.153–§11.4.159)

- `docs/features/Status.md` + `Status_Summary.md` MUST always reflect every feature with per-row video-recording confirmation (HTML+PDF+DOCX exports in sync via `docs_chain`). Code-present features missing from the table are §11.4.153 violations.
- All recordings MUST be window-scoped (not whole-desktop) per §11.4.154. A new recording run removes its own prior in-scope recordings first.
- All recording filenames MUST start with `<PREFIX>---` per §11.4.155 (triple-hyphen separator, canonical form `<PREFIX>---<feature-or-scope>---<run-id>.<ext>`). `PREFIX` is resolved from `HELIX_RELEASE_PREFIX` env var or the lowercased project-root dir name.
- **Recording path override:** All recordings use `$HOME/Downloads` as the default save path per §11.4.158(D). This project does NOT override the default — no separate recording-path variable is defined. Recordings at non-`$HOME/Downloads` paths (e.g. `/Volumes/T7/Downloads/Recordings/`) are a temporary convenience and MUST be migrated before release tagging.
- All CI/CD automation is DISABLED per §11.4.156. No active `.yaml` workflow files exist; enforcement is local-only via `pre_build_verification.sh`.
- GEMINI.md lockstep per §11.4.157 — all per-agent context carriers carry the same highest §11.4.N anchor.
- Every recording's on-screen content MUST be machine-read and verified as a genuine working result per §11.4.158. A video without read-the-screen verification is not evidence.
- §11.4.159 — window-specific MP4 with vision validation, expected-content specification before recording, content-verification workflow (`SPECIFY→RECORD→EXTRACT→VERIFY→CHECK→ACCEPT`), terminal cleanup per window id, and root cause analysis on rejected recordings.

Project-specific architecture notes

- Persistence seam: `server/internal/store.Repository`. MVP wires the in-memory implementation; the `pgx/PostgreSQL` implementation is the production target (`architecture.md` §4).
 - Trust boundary (security): the artifact-signature verification key comes ONLY from server configuration — never from the request (`server/internal/api/handlers_artifact.go:resolvePublicKey`).
-

Project overrides of universal rules

(none — this project does not override any universal clause)